
Especificación de requisitos de software

Proyecto:
Revisión

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

IEEE Std 830-1998

Plataforma de Servicios, Compras y Saldo Digital del Campus

Módulo 4.1 – Gestión de Usuarios y Seguridad | Módulo 4.10 – Identificación por Tarjeta Universitaria (RFID/NFC)

Curso	Ingeniería de Software (SCD-1011)
Equipo	Equipo No. 6 – Parcial 2
Módulos asignados	4.1 y 4.10
Revisión	1.1
Fecha	Abril 2026
Estado	Versión 1.0

Integrantes del equipo de desarrollo

#	Nombre completo
1	Silva López Irving Javier
2	Soto Trejo Ángel
3	Morales Martínez Adrián
4	Pacheco García Jesús Adrián
5	Paz Rodríguez Diego Alejandro

Documento validado por las partes en fecha: _____

Por el cliente / Docente	Por el equipo de desarrollo
Fdo. _____	Fdo. _____

Contenido

<u>FICHA DEL DOCUMENTO</u>	3
<u>CONTENIDO</u>	4
<u>1 INTRODUCCIÓN</u>	6
<u>1.1 Propósito</u>	6
<u>1.2 Alcance</u>	6
<u>1.3 Personal involucrado</u>	6
<u>1.4 Definiciones, acrónimos y abreviaturas</u>	6
<u>1.5 Referencias</u>	6
<u>1.6 Resumen</u>	6
<u>2 DESCRIPCIÓN GENERAL</u>	7
<u>2.1 Perspectiva del producto</u>	7
<u>2.2 Funcionalidad del producto</u>	7
<u>2.3 Características de los usuarios</u>	7
<u>2.4 Restricciones</u>	7
<u>2.5 Suposiciones y dependencias</u>	7
<u>2.6 Evolución previsible del sistema</u>	7
<u>3 REQUISITOS ESPECÍFICOS</u>	7
<u>3.1 Requisitos comunes de los interfaces</u>	8
<u>3.1.1 Interfaces de usuario</u>	8
<u>3.1.2 Interfaces de hardware</u>	8
<u>3.1.3 Interfaces de software</u>	8
<u>3.1.4 Interfaces de comunicación</u>	8
<u>3.2 Requisitos funcionales</u>	8
<u>3.2.1 Requisito funcional 1</u>	9

3.2.2	Requisito funcional 2	9
3.2.3	Requisito funcional 3	9
3.2.4	Requisito funcional n	9
3.3	Requisitos no funcionales	9
3.3.1	Requisitos de rendimiento	9
3.3.2	Seguridad	9
3.3.3	Fiabilidad	9
3.3.4	Disponibilidad	9
3.3.5	Mantenibilidad	10
3.3.6	Portabilidad	10
3.4	Otros requisitos	10
4	APÉNDICES	10

1. Introducción

Este documento constituye la Especificación de Requisitos de Software (SRS) para los módulos 4.1 – Gestión de Usuarios y Seguridad y 4.10 – Identificación por Tarjeta Universitaria (RFID/NFC), pertenecientes al proyecto integrador de la asignatura Ingeniería de Software (SCD-1011). El documento ha sido elaborado por el Equipo No. 6 conforme al estándar IEEE Std 830-1998.

1.1 Propósito

El propósito principal de este documento es definir, de manera completa, consistente y verificable, los requisitos funcionales y no funcionales de los módulos asignados, con los siguientes objetivos:

- Dar al equipo de desarrollo una base sólida para el diseño e implementación.
- Permitir al equipo de pruebas planificar y ejecutar las validaciones correspondientes.
- Proporcionar a los equipos de evaluación cruzada criterios claros para su revisión.
- Que las partes interesadas (docentes, alumnos y personal del campus) comprendan el alcance y las capacidades del sistema.

La audiencia principal son los integrantes del Equipo 6, los docentes evaluadores de SCD-1011 y los equipos asignados a la evaluación cruzada de estos módulos.

1.2 Alcance

El sistema global se denomina Plataforma de Servicios, Compras y Saldo Digital del Campus y está compuesto por diez módulos desarrollados en paralelo por diferentes equipos. Este documento cubre únicamente los módulos asignados al Equipo 6:

- Módulo 4.1 – Gestión de Usuarios y Seguridad: gestiona el ciclo de vida completo de los usuarios del sistema (registro, autenticación, roles, permisos, perfil y bitácora de actividad). Es la columna vertebral de seguridad de toda la plataforma.
- Módulo 4.10 – Identificación por Tarjeta Universitaria (RFID/NFC): extiende la identidad digital al mundo físico, asociando tarjetas universitarias físicas a cuentas de usuario para autenticación por proximidad, consulta de saldo y confirmación de consumos.

Ambos módulos se implementan con el stack tecnológico oficial: Laravel (backend) + Inertia.js + Vue 3 (frontend) + Axios (HTTP). El módulo 4.10 depende directamente del módulo 4.1 para la asociación usuario-tarjeta y gestión de sesiones.

1.3 Personal involucrado

Campo	Valor	Detalle
Nombre del equipo	Equipo No. 6	Parcial 2 – SCD-1011
Integrantes	5 alumnos	Silva López Irving J., Soto Trejo Ángel, Morales Martínez Adrián, Pacheco García Jesús A., Paz Rodríguez Diego A.

Rol	Desarrolladores	Análisis, diseño, implementación, pruebas y documentación de los módulos 4.1 y 4.10
Categoría	Estudiantes	Ingeniería en Sistemas / Computación
Contacto	Correo institucional	Correo institucional de cada integrante del equipo
Aprobación	Docente	Titular de la asignatura SCD-1011

1.4 Definiciones, acrónimos y abreviaturas

Término	Definición
SRS	Software Requirements Specification / Especificación de Requisitos de Software.
RFID	Radio Frequency Identification – tecnología de identificación por radiofrecuencia.
NFC	Near Field Communication – comunicación de campo cercano, variante de RFID a corta distancia.
UID	Unique Identifier – identificador único grabado en el chip de la tarjeta universitaria.
Laravel	Framework PHP utilizado para el desarrollo del backend de la plataforma.
Inertia.js	Librería que conecta el backend Laravel con el frontend Vue 3 sin necesidad de una API REST explícita.
Vue 3	Framework JavaScript para el desarrollo de interfaces de usuario reactivas (frontend).
Axios	Cliente HTTP para el consumo de servicios y APIs desde el frontend Vue 3.
PHPUnit	Framework de pruebas unitarias e integración para PHP, utilizado en el backend.
CSV	Comma-Separated Values – formato de exportación de datos tabulares.
PDF	Portable Document Format – formato de exportación de reportes imprimibles.
QR	Quick Response Code – código de respuesta rápida utilizado para folios de entrega.
Dashboard	Pantalla principal de resumen con indicadores clave del módulo.
Rol	Conjunto de permisos y accesos asignados a un tipo de usuario del sistema.
Permiso	Clave granular que autoriza una operación específica dentro del sistema (ej. user.write).

SLA	Service Level Agreement – acuerdo de nivel de servicio que define tiempos de respuesta mínimos.
ORM	Object-Relational Mapping – Eloquent en Laravel, abstrae las consultas SQL.
CSRF	Cross-Site Request Forgery – ataque web mitigado mediante tokens en formularios.
HTTPS	HyperText Transfer Protocol Secure – protocolo de comunicación cifrada cliente-servidor.
Soft delete	Eliminación lógica de registros: se marcan como eliminados sin borrarlos físicamente de la base de datos.

1.5 Referencias

Ref.	Título	URL / Ruta	Fecha	Autor
[R1]	IEEE Std 830-1998 – Recommended Practice for SRS	IEEE Standards	1998	IEEE
[R2]	Laravel Documentation v11.x	https://laravel.com/docs	2024-2026	Laravel LLC
[R3]	Vue 3 – Documentación oficial	https://vuejs.org	2024-2026	Evan You
[R4]	Descripción del Proyecto Integrador SCD-1011	ALL_MODULES.txt	2025	Equipo Docente
[R5]	API REST – Campus Digital, Documentación Técnica v1.0	API_REST_CampusDigital_v1.0.pdf	2026	Equipo 6
[R6]	PHPUnit Documentation	https://phpunit.de/documentation	2024	Sebastian Bergmann

1.6 Resumen del documento

Este documento se organiza en cuatro secciones principales:

- Sección 1 – Introducción: propósito, alcance, personal involucrado, glosario y referencias.
- Sección 2 – Descripción general: perspectiva del producto, funcionalidad, características de los usuarios, restricciones, suposiciones y evolución previsible del sistema.
- Sección 3 – Requisitos específicos: interfaces de usuario, hardware, software y comunicación; requisitos funcionales detallados de los módulos 4.1 y 4.10; requisitos no funcionales.
- Sección 4 – Apéndices: dashboards e indicadores por módulo, catálogo de reportes y mapa de cobertura de pruebas unitarias.

2. Descripción general

2.1 Perspectiva del producto

La Plataforma de Servicios, Compras y Saldo Digital del Campus es un sistema modular compuesto por diez módulos independientes pero interconectados, desarrollados en paralelo por diferentes equipos de la asignatura. Los módulos 4.1 y 4.10, asignados al Equipo 6, actúan como la columna vertebral de seguridad e identidad de toda la plataforma:

- El Módulo 4.1 es el punto de entrada al sistema. Todos los demás módulos dependen de él para autenticar usuarios y verificar permisos antes de ejecutar cualquier operación. Sin este módulo, el resto de la plataforma no puede operar de forma segura.
- El Módulo 4.10 extiende la autenticación al mundo físico. Permite que los usuarios interactúen con el sistema mediante su tarjeta universitaria RFID/NFC, habilitando accesos rápidos en puntos de servicio como cafetería, copiadora y souvenir sin necesidad de ingresar credenciales manualmente.

Ambos módulos se ejecutan en el servidor web del campus y son accesibles a través de navegadores modernos. El módulo 4.10 adicionalmente se comunica con lectores RFID/NFC instalados en los puntos de servicio del campus, o bien mediante un simulador por software cuando no se dispone del hardware físico.

2.2 Funcionalidad del producto

Módulo 4.1 – Gestión de Usuarios y Seguridad

- Registro, edición y baja lógica (soft delete) de cuentas de usuario desde el panel de administración.
- Autenticación segura con correo institucional y contraseña, con bloqueo automático tras intentos fallidos.
- Verificación de correo electrónico al crear la cuenta (envío de enlace de verificación).
- Gestión de roles: Estudiante, Proveedor/Área y Administrador, con asignación y revocación desde la interfaz.
- Permisos granulares por rol (user.read, user.write, audit.read, role.write, etc.) verificados por middleware en backend y ocultados en frontend según el rol.
- Perfil de usuario editable: nombre, foto de perfil, datos personales extendidos (fecha de nacimiento, género, dirección).
- Bitácora de accesos (AccesoBitacora) y de actividad del sistema (ActividadBitacora) con registro automático de cada acción relevante.
- Dashboard con métricas de usuarios activos, distribución por roles e intentos de acceso.
- Reportes exportables en CSV y PDF sobre usuarios, accesos y actividad del sistema.
- Gestión de sesiones: registro de cada sesión iniciada (IP, user-agent, tiempo de inicio/expiración), cierre e invalidación de sesión.

Módulo 4.10 – Identificación por Tarjeta Universitaria (RFID/NFC)

- Registro y asociación del UID de una tarjeta física a una cuenta de usuario existente.
- Autenticación por proximidad (real o simulada) mediante la lectura del UID; integración con el módulo 4.1 para iniciar sesión.
- Autenticación alternativa mediante UID + PIN para sistemas embebidos o de hardware externo (endpoint /rfid/auth).
- Consulta rápida de saldo del monedero universitario al identificar al usuario con la tarjeta (integración con módulo 4.2).
- Confirmación de consumos y entregas: flujo de dos lecturas (identificación + confirmación) con generación de folio.
- Bloqueo y desbloqueo de tarjetas con registro de motivo y usuario responsable.
- Auditoría completa de lecturas: cada uso de tarjeta queda registrado con UID, módulo, tipo de operación, resultado y timestamp.
- Dashboard de lecturas del día, uso por módulo y tarjetas bloqueadas activas.
- Reportes de lecturas por usuario, uso por módulo e incidentes de seguridad, exportables en CSV y PDF.

2.3 Características de los usuarios

Tipo de usuario	Descripción	Funciones principales en los módulos 4.1 y 4.10
Estudiante	Alumno universitario con cuenta activa	Autenticarse (credenciales o tarjeta), consultar saldo, ver su historial de movimientos y pedidos, editar su perfil.
Proveedor / Área	Personal de cafetería, copias, souvenirs u otras áreas del campus	Confirmar entregas de pedidos mediante lectura de tarjeta, consultar saldo (solo lectura), actualizar estado de pedidos.
Administrador	Personal de TI o administración del campus	Gestionar usuarios, roles, permisos, tarjetas, bitácoras, reportes y toda la configuración del sistema.
Operador de punto de lectura	Personal en mostrador con lector RFID/NFC	Identificar al usuario por tarjeta, confirmar consumos, reportar incidentes con tarjetas perdidas o bloqueadas.

2.4 Restricciones

- El sistema debe desarrollarse obligatoriamente con el stack tecnológico oficial: Laravel (backend), Inertia.js + Vue 3 (frontend) y Axios (HTTP).
- Las pruebas unitarias deben implementarse con PHPUnit; se acepta Pest como alternativa sobre PHPUnit.
- La base de datos debe ser relacional: MySQL 8.0 o PostgreSQL 15, según lo defina el equipo o el docente.

- Los reportes deben ser exportables al menos en formato CSV; la exportación en PDF es obligatoria para módulos 4.1 y 4.10 (ya implementada en el sistema actual).
- La autenticación RFID/NFC puede simularse por software (campo de texto con UID) cuando no se dispone de hardware físico en el sprint; el sistema implementa este simulador en las rutas /simulador/*.
- El sistema debe ser accesible desde navegadores modernos: Chrome, Firefox y Edge en sus versiones más recientes.
- El tiempo de respuesta objetivo para operaciones de autenticación es menor a 2 segundos bajo condiciones normales.
- Todos los datos personales deben tratarse conforme a la LFPDPPP (Ley Federal de Protección de Datos Personales en Posesión de los Particulares) de México.
- La interfaz de usuario debe estar completamente en español (México), incluyendo mensajes de error, etiquetas y reportes.

2.5 Suposiciones y dependencias

- Se asume que existe una base de datos relacional compartida entre todos los módulos del proyecto integrador.
- Se asume que el módulo 4.2 (Saldo Digital Universitario) provee los servicios de saldo (SaldoMonedero, SaldoMovimiento), consumidos internamente por el módulo 4.10 a través de la API REST del proyecto.
- Se asume que los lectores RFID/NFC físicos devuelven el UID de la tarjeta como cadena hexadecimal en mayúsculas (ej. A1B2C3D4).
- El módulo 4.10 depende del módulo 4.1 para la gestión de cuentas de usuario, autenticación y verificación de permisos.
- Se asume que el entorno de producción cuenta con HTTPS habilitado (certificado TLS) para proteger todas las comunicaciones.
- El módulo 4.5 (Pedidos y Seguimiento) recibe notificaciones de entrega confirmadas desde el módulo 4.10 al confirmar un pedido con tarjeta.

2.6 Evolución previsible del sistema

- Integración con directorio LDAP/Active Directory del campus para autenticación federada (SSO institucional).
- Implementación de autenticación multifactor (MFA) con OTP por correo electrónico o aplicación autenticadora.
- Soporte para múltiples tarjetas por usuario: tarjeta principal y tarjeta de respaldo en caso de pérdida.
- Módulo de self-service para que el usuario reporte y solicite reemplazo de su tarjeta directamente desde la plataforma.
- Ampliación del módulo RFID/NFC para registrar asistencia en aulas y laboratorios del campus.
- Exportación de reportes en formato XLSX (Excel) además de CSV y PDF.
- Implementación de notificaciones en tiempo real (push/websocket) para eventos de seguridad críticos como bloqueos de cuenta o tarjeta.

- Panel de auditoría centralizado que consolide bitácoras de todos los módulos del sistema.

3. Requisitos específicos

Esta sección contiene la descripción detallada de todos los requisitos funcionales y no funcionales de los módulos 4.1 y 4.10. Cada requisito funcional se presenta mediante una tabla normalizada con número identificador, nombre, tipo, fuente, prioridad y descripción. La numeración de los requisitos del módulo 4.10 utiliza el prefijo RF-10-XX para distinguirlos de los del módulo 4.1.

3.1 Requisitos comunes de las interfaces

A continuación se describen los requisitos de las interfaces de entrada y salida que aplican a ambos módulos.

3.1.1 Interfaces de usuario

- Diseño responsivo compatible con resoluciones de escritorio (1280×720 o superior) y dispositivos móviles (360px en adelante).
- Paleta de colores institucional definida por el equipo del proyecto; interfaz construida con Vue 3 + Inertia.js.
- Validaciones en tiempo real en todos los formularios: feedback visual inmediato al usuario (bordes rojos, mensajes de error debajo del campo).
- Mensajes de error descriptivos y orientados a la acción del usuario (ej. 'El correo electrónico ya está registrado. Intenta iniciar sesión o recupera tu contraseña.').
- Navegación mediante menú lateral o superior con secciones visibles solo según el rol del usuario autenticado; los elementos sin permiso se ocultan en el frontend.
- Dashboard accesible como página de inicio tras autenticación exitosa.
- Interfaz completamente en español (México): etiquetas, mensajes de error, botones y contenido de reportes.

3.1.2 Interfaces de hardware

- Módulo 4.10: el sistema debe ser compatible con lectores RFID/NFC que transmitan el UID de la tarjeta vía USB (emulación de teclado HID) o mediante API serial.
- En ausencia de hardware físico, el sistema debe ofrecer un modo de simulación que permita ingresar el UID manualmente desde la interfaz web (ya implementado en las rutas /simulador/uid y /simulador/login-rfid).
- Módulo 4.1: no requiere hardware especializado más allá de teclado, pantalla y conexión de red estándar.

3.1.3 Interfaces de software

- Backend: Laravel 11 con PHP 8.2 o superior; manejo de autenticación con Laravel Fortify.
- Frontend: Vue 3 con Inertia.js; componentes construidos con Composition API. Comunicación interna mediante Inertia y consumo de endpoints externos mediante Axios.
- Base de datos: MySQL 8.0 o PostgreSQL 15 (relacional); migraciones gestionadas con el sistema de migraciones de Laravel.
- PHPUnit para pruebas unitarias e integración en el backend.

- El módulo 4.10 consume internamente el servicio de consulta de saldo del módulo 4.2 a través de los endpoints /saldo-monederos/usuario/{id} y /saldo-movimientos.

3.1.4 Interfaces de comunicación

- Protocolo HTTPS/TLS 1.2 o superior para toda comunicación cliente-servidor.
- API REST interna del proyecto: autenticación mediante header X-API-KEY; formato JSON; paginación estándar de 15 registros por página.
- El módulo 4.10 se comunica con lectores RFID mediante polling HTTP (endpoints /simulador/*) cuando opera en modo simulación; en modo real, recibe el UID directamente desde el hardware.
- Las notificaciones internas pueden implementarse mediante polling periódico o websockets (Laravel Echo + Pusher/Reverb) según el avance del proyecto.

3.2 Requisitos funcionales – Módulo 4.1

Número de requisito	RF-01
Nombre de requisito	Registro de usuario
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema permite al Administrador crear nuevas cuentas de usuario desde el panel de administración (/admin/usuarios/create). Los campos requeridos son: nombre, apellido, correo institucional (único en la BD), contraseña (con confirmación, mínimo 8 caracteres), teléfono (opcional) y rol inicial. El modelo Usuario implementa SoftDeletes para baja lógica. Al crear la cuenta, el sistema envía un correo de verificación de email (VerifyEmailNotification). El campo email_verificado se actualiza a true una vez que el usuario confirma su correo. El sistema valida unicidad de email antes de insertar el registro.
Número de requisito	RF-02
Nombre de requisito	Autenticación de usuarios
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema provee un formulario de inicio de sesión con campos de correo electrónico y contraseña. La autenticación se gestiona mediante Laravel Fortify con el campo password_hash (getAuthPassword()). Tras autenticación exitosa, el sistema redirige al usuario al Dashboard

	correspondiente a su rol. Tras un número configurable de intentos fallidos consecutivos, la cuenta se bloquea temporalmente (campos bloqueado y bloqueado_hasta en el modelo Usuario); el método estaBloqueado() verifica automáticamente si el bloqueo temporal venció. Cada intento de acceso (exitoso o fallido) se registra en la tabla acceso_bitacora con IP, user-agent, evento y timestamp.
Número de requisito	RF-03
Nombre de requisito	Gestión de roles y permisos
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema soporta roles definidos en la tabla 'rol' (Administrador, Estudiante, Proveedor y otros según configuración). La relación usuario-rol es muchos-a-muchos (tabla usuario_rol con soft delete en pivot). Los permisos se definen en la tabla 'permiso' con clave granular (ej. user.read, user.write, audit.read, card.block, report.cards). La relación rol-permiso es muchos-a-muchos (tabla rol_permiso). Los permisos se verifican en backend mediante middleware 'permission:{clave}' y en frontend los elementos de UI no autorizados se ocultan según el rol activo. El Administrador puede asignar y revocar roles y permisos desde las secciones /admin/roles y /admin/permisos.
Número de requisito	RF-04
Nombre de requisito	Perfil de usuario y administración básica
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	Cada usuario puede consultar y editar su perfil desde /perfil: nombre, apellido, teléfono y foto de perfil (upload con almacenamiento local). El cambio de contraseña requiere ingresar la contraseña actual. Los datos personales extendidos (fecha de nacimiento, género, dirección, preferencias) se almacenan en la tabla usuario_perfil (modelo UsuarioPerfil, relación hasOne). El Administrador puede listar, filtrar, ver detalle, editar, bloquear/desbloquear y eliminar (soft delete) cuentas de usuario desde /admin/usuarios. Todas estas acciones administrativas quedan registradas en la bitácora de actividad.
Número de requisito	RF-05

Nombre de requisito	Cierre de sesión y manejo de sesiones
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema provee un botón de cierre de sesión accesible desde cualquier pantalla. Las sesiones activas se registran en la tabla usuario_sesion (modelo UsuarioSesion) con IP, user-agent, fecha de inicio, fecha de expiración y estado activo/inactivo. Al cerrar sesión, el token de sesión se invalida en el servidor y el campo termina_at se actualiza. La sesión expira automáticamente según la configuración de Laravel. Al expirar o cerrar sesión, el usuario es redirigido a la pantalla de inicio de sesión. El sistema soporta consulta de sesiones activas y cerradas desde la API REST (/sesiones).
Número de requisito	RF-06
Nombre de requisito	Bitácora de accesos y actividad
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema registra automáticamente dos tipos de bitácora: (1) AccesoBitacora: registra eventos de autenticación (login_success, login_failed, logout, registro) con campos usuario_id, sesion_id, email_intentado, evento, exito, ip, user_agent, detalle y meta_json. (2) ActividadBitacora: registra acciones del sistema (crear_usuario, editar_usuario, cambiar_rol, bloquear_tarjeta, etc.) con campos accion, modulo, target_tabla, target_id, exito, ip, user_agent y meta_json. Ambas bitácoras usan SoftDeletes. El Administrador puede consultarlas, filtrarlas y exportarlas (CSV y PDF) desde /admin/bitacora/accesos y /admin/bitacora/actividad.
Número de requisito	RF-07
Nombre de requisito	Dashboard del módulo 4.1
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El módulo presenta un dashboard accesible en /dashboard con los siguientes indicadores: (1) total de usuarios activos vs inactivos en el

	sistema, (2) distribución de usuarios por rol (gráfica o tabla), (3) intentos de acceso exitosos vs fallidos en los últimos 7 días, (4) lista de actividad reciente (últimas 10 entradas de la bitácora). Los datos son cargados desde el DashboardController y servidos como props de Inertia a los componentes Vue 3 del dashboard.
Número de requisito	RF-08
Nombre de requisito	Reportes del módulo 4.1
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El módulo genera reportes accesibles en /admin/reportes y /admin/bitacora, todos con filtros y exportación en CSV y PDF: (1) Usuarios por rol: listado de todos los usuarios agrupados por rol con estado de cuenta (activo/bloqueado) y fecha de último acceso; (2) Accesos por periodo: cantidad de inicios de sesión exitosos y fallidos en un rango de fechas configurable; (3) Historial de actividad: bitácora filtrable por usuario, módulo, tipo de acción y rango de fechas; (4) Accesos por periodo con desglose (ReporteController); (5) Exportación de bitácora de accesos por módulo (BitacoraController). Todos los reportes se visualizan en tablas filtrables dentro de la plataforma y son exportables en CSV y PDF.

3.3 Requisitos funcionales – Módulo 4.10

Número de requisito	RF-10-01
Nombre de requisito	Registro y asociación de tarjeta universitaria
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El Administrador puede registrar nuevas tarjetas universitarias desde /admin/tarjetas/create (TarjetaController). El proceso permite ingresar el UID manualmente o capturarlo desde un lector RFID (modo simulado mediante Cache). El sistema valida que el UID sea único en la tabla tarjeta_universitaria. La tarjeta se asocia a un usuario existente (campo usuario_id). La operación queda registrada en la bitácora con fecha, hora y usuario que la realizó. La API REST expone este registro en POST /tarjetas con validación de UID duplicado (error 422).

Número de requisito	RF-10-02
Nombre de requisito	Autenticación por proximidad (real o simulada)
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema permite autenticar un usuario presentando su tarjeta universitaria ante un lector RFID/NFC en /lector (TarjetaLecturaController). Al detectar el UID, el sistema: (1) busca la tarjeta asociada, (2) verifica que no esté bloqueada, (3) inicia la sesión del usuario con los permisos de su rol (RfidLoginController). El modo simulado funciona mediante el endpoint /simulador/uid (Cache) que pasa el UID a la pantalla de lector sin hardware físico. La autenticación alternativa para sistemas embebidos se realiza mediante POST /rfid/auth con UID + PIN (RfidApiController), ruta pública que no requiere API Key.
Número de requisito	RF-10-03
Nombre de requisito	Consulta rápida de saldo mediante tarjeta
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	Al identificar a un usuario por tarjeta en el punto de lectura, el sistema muestra inmediatamente: nombre completo del usuario, foto de perfil (si existe), saldo actual del monedero universitario y estado de la tarjeta (activa/bloqueada). El saldo se obtiene consultando el servicio interno del módulo 4.2 (endpoint GET /rfid/saldo/{uid} del RfidApiController). Esta información se presenta en la pantalla del operador en menos de 2 segundos tras la lectura.
Número de requisito	RF-10-04
Nombre de requisito	Confirmación de consumos y entregas con tarjeta
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema permite confirmar la entrega de un pedido mediante lectura de tarjeta desde /lector (TarjetaLecturaController, método

	confirmarPedido). El flujo implementado es: (1) el operador identifica al usuario por tarjeta, (2) el sistema muestra los pedidos pendientes del usuario (GET /rfid/pedidos/{uid}), (3) el operador selecciona el pedido a entregar, (4) el usuario acerca la tarjeta nuevamente para confirmar (segunda lectura), (5) el sistema registra la entrega y actualiza el estado del pedido mediante POST /pedidos/{id}/confirmar-tarjeta. La operación genera un folio de entrega registrado en tarjeta_lectura con tipo_lectura = 'confirmacion_entrega'.
Número de requisito	RF-10-05
Nombre de requisito	Bloqueo y desbloqueo de tarjetas
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El Administrador puede bloquear y desbloquear tarjetas desde /admin/tarjetas/{tarjeta}/toggle-block (TarjetaController, permiso card.block). Al bloquear se registra el motivo del bloqueo y el usuario responsable. Una tarjeta bloqueada impide autenticación y confirmación de consumos (validación en RfidApiController y TarjetaLecturaController). El sistema notifica al operador/usuario cuando intenta usar una tarjeta bloqueada, mostrando el motivo. La API REST expone bloqueo en POST /tarjetas/{id}/bloquear y desbloqueo en POST /tarjetas/{id}/desbloquear. Cada evento se registra en la bitácora de actividad.
Número de requisito	RF-10-06
Nombre de requisito	Auditoría y registro de lecturas de tarjetas
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El sistema mantiene un registro completo en la tabla tarjeta_lectura de cada uso de tarjeta, incluyendo: uid_leido, tarjeta_id, modulo (cafeteria, copias, souvenirs, acceso, etc.), tipo_lectura (acceso, consumo, consulta_saldo, confirmacion_entrega), exito (booleano), operador_usuario_id, pedido_id (si aplica) y timestamp. El Administrador consulta este registro desde /admin/tarjetas/dashboard y /admin/tarjetas/reportes (TarjetaReporteController) con filtros por usuario, fecha, módulo y resultado. Todos los reportes son exportables en CSV y PDF.

Número de requisito	RF-10-07
Nombre de requisito	Dashboard del módulo 4.10
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El módulo presenta un dashboard accesible en /admin/tarjetas/dashboard (TarjetaDashboardController) con los siguientes indicadores: (1) total de lecturas de tarjeta registradas en la fecha actual, (2) desglose de uso por módulo del sistema (autenticación, saldo, entregas, otros), (3) número de tarjetas con estado bloqueado activas en ese momento, (4) top 5 usuarios con mayor número de lecturas en el periodo. Los indicadores son servidos como props Inertia desde el controlador y visualizados en componentes Vue 3.
Número de requisito	RF-10-08
Nombre de requisito	Reportes del módulo 4.10
Tipo	Requisito funcional
Fuente	Equipo docente / Especificación del proyecto SCD-1011
Prioridad	Alta / Esencial
Descripción	El módulo genera los siguientes reportes en /admin/tarjetas/reportes (TarjetaReporteController): (1) Lecturas por usuario: historial de todas las lecturas de tarjeta de un usuario en un periodo seleccionado (exportable en CSV y PDF); (2) Uso por módulo: conteo de lecturas agrupadas por módulo del sistema y tipo de operación en un periodo (CSV y PDF); (3) Incidentes y auditoría: listado de lecturas fallidas, tarjetas bloqueadas y eventos de seguridad relevantes en un periodo (CSV y PDF). Todos los reportes son filtrables por fecha desde la interfaz y exportables mediante rutas dedicadas en TarjetaReporteController.

3.4 Requisitos no funcionales

3.4.1 Rendimiento

- El 95% de las operaciones de autenticación (por credenciales o por tarjeta RFID) deben completarse en menos de 2 segundos bajo condiciones normales de operación de la red del campus.
- El sistema debe soportar al menos 50 usuarios autenticados simultáneamente sin degradación perceptible del rendimiento.
- La generación de reportes con hasta 10,000 registros debe completarse en menos de 10 segundos.

- El dashboard de ambos módulos debe cargarse en menos de 3 segundos en una conexión de red estándar.
- La lectura de tarjeta RFID/NFC (desde la detección del UID hasta mostrar la información del usuario) debe completarse en menos de 3 segundos.

3.4.2 Seguridad

- Las contraseñas deben almacenarse con hash bcrypt (costo mínimo 10) en el campo password_hash; nunca en texto plano. Laravel Fortify gestiona este proceso automáticamente.
- Toda comunicación cliente-servidor debe realizarse sobre HTTPS/TLS 1.2 o superior.
- El sistema debe implementar protección contra CSRF en todos los formularios mediante el token CSRF de Laravel (@csrf / X-XSRF-TOKEN).
- La protección contra inyección SQL se garantiza mediante el ORM Eloquent de Laravel, evitando consultas SQL crudas sin parámetros ligados.
- El acceso a la bitácora y al módulo de auditoría de tarjetas está restringido al rol Administrador (middleware 'role:administrador' + 'permission:audit.read').
- Los UIDs de tarjeta deben almacenarse de forma segura; la API los recibe y almacena en la base de datos; el acceso al campo uid está protegido por la capa de autenticación de la API (X-API-KEY).
- Las sesiones se manejan mediante el sistema de sesiones firmadas de Laravel; los tokens de sesión se invalidan al cerrar sesión.

3.4.3 Fiabilidad

- El sistema debe tener una disponibilidad mínima del 99% durante el horario de operación del campus (07:00 – 22:00, lunes a sábado).
- En caso de fallo en la lectura RFID/NFC, el sistema debe ofrecer un modo de recuperación: ingreso manual del UID (simulador) o autenticación alternativa por credenciales convencionales.
- El sistema debe implementar transacciones de base de datos (DB::transaction) para garantizar la integridad de los registros de bitácora y auditoría de tarjetas.
- Los registros de bitácora no deben poder editarse ni eliminarse físicamente por ningún rol del sistema (solo soft delete disponible para administradores).

3.4.4 Disponibilidad

- Disponibilidad objetivo: 99% en horario de operación (07:00 – 22:00, lunes a sábado).
- El tiempo máximo de mantenimiento planificado es de 2 horas semanales, a realizarse fuera del horario de operación.
- El sistema debe mostrar una página de mantenimiento informativa durante las ventanas de mantenimiento programadas.

3.4.5 Mantenibilidad

- El código backend debe seguir las convenciones de estilo Laravel / PSR-12. El código frontend debe seguir las convenciones de Vue 3 con Composition API.
- Cada módulo debe contar con pruebas unitarias en PHPUnit que cubran al menos las reglas de negocio principales y las validaciones de formulario (mínimo según el Apéndice D).

- El sistema debe contar con un archivo README actualizado con instrucciones de instalación, configuración y ejecución de pruebas.
- Todos los cambios en el esquema de base de datos deben gestionarse mediante migraciones de Laravel para garantizar trazabilidad y reproducibilidad del entorno.

3.4.6 Portabilidad

- El sistema debe funcionar correctamente en Chrome, Firefox y Edge en sus versiones más recientes (escritorio y móvil).
- El backend debe poder desplegarse en servidores Linux (Ubuntu 22.04 LTS o superior) con PHP 8.2+, Nginx o Apache.
- El uso del ORM Eloquent debe minimizar la dependencia de instrucciones SQL específicas de un motor, facilitando la migración entre MySQL y PostgreSQL.

3.5 Otros requisitos

- Evaluación cruzada: cada sprint, un equipo diferente debe revisar la UI, las pruebas unitarias y los reportes de los módulos 4.1 y 4.10, registrando sus hallazgos en el formato establecido por el docente.
- Documentación de sprint: el equipo debe mantener actualizada la bitácora del sprint (user stories completadas, impedimentos encontrados y velocidad del equipo).
- Requisito legal: el sistema debe tratar los datos personales conforme a la LFPDPPP de México. Los datos personales no deben exponerse en logs ni reportes sin protección de acceso.
- Requisito cultural: la interfaz debe estar completamente en español (México), incluyendo mensajes de error, etiquetas de formulario y contenido de reportes exportados.

4. Apéndices

Apéndice A – Dashboard e indicadores: Módulo 4.1

Indicador / KPI	Descripción y fuente de datos
Usuarios activos	Total de usuarios con estado activo (bloqueado=false, deleted_at IS NULL) al momento de la consulta. Fuente: tabla 'usuario'.
Usuarios inactivos / bloqueados	Usuarios con bloqueado=true o deleted_at IS NOT NULL. Permite detectar cuentas suspendidas.
Distribución por rol	Gráfica o tabla con conteo de usuarios agrupados por rol (Estudiante, Proveedor, Administrador). Fuente: tabla 'usuario_rol' + 'rol'.
Intentos de acceso exitosos (7d)	Número de inicios de sesión con éxito=true en los últimos 7 días. Fuente: tabla 'acceso_bitacora', evento='login_success'.
Intentos de acceso fallidos (7d)	Número de intentos con éxito=false en los últimos 7 días. Útil para detectar ataques de fuerza bruta. Fuente: tabla 'acceso_bitacora', evento='login_failed'.
Actividad reciente	Tabla con las últimas 10 entradas de ActividadBitacora: acción, usuario, módulo, fecha/hora. Fuente: tabla 'actividad_bitacora'.
Sesiones activas	Número de sesiones con activa=true en este momento. Fuente: tabla 'usuario_sesion'.

Apéndice B – Dashboard e indicadores: Módulo 4.10

Indicador / KPI	Descripción y fuente de datos
Lecturas del día	Total de lecturas de tarjeta registradas en la fecha actual (created_at = hoy). Fuente: tabla 'tarjeta_lectura'.
Uso por tipo de operación	Desglose de lecturas por tipo_lectura: acceso, consumo, consulta_saldo, confirmacion_entrega. Fuente: tabla 'tarjeta_lectura'.
Uso por módulo del campus	Desglose de lecturas agrupadas por módulo (cafeteria, copias, souvenirs, biblioteca, acceso, otro). Fuente: tabla 'tarjeta_lectura'.
Tarjetas bloqueadas activas	Número de tarjetas con estado='bloqueada' en este momento. Fuente: tabla 'tarjeta_universitaria'.
Top 5 usuarios más activos	Ranking de los 5 usuarios con mayor número de lecturas en el periodo (día/semana). Fuente: tabla 'tarjeta_lectura' agrupada

	por tarjeta_id / usuario.
Lecturas fallidas del día	Total de lecturas con exito=false en la fecha actual: UIDs inválidos, tarjetas bloqueadas o errores de lector. Fuente: tabla 'tarjeta_lectura'.

Apéndice C – Catálogo de reportes

ID	Nombre	Descripción	Módulo	Formato	Ruta
R-01	Usuarios por rol	Listado de todos los usuarios agrupados por rol, con estado de cuenta (activo/bloqueado) y fecha de último acceso.	4.1	CSV / PDF	/admin/usuarios/export
R-02	Accesos por periodo	Conteo de inicios de sesión exitosos y fallidos en un rango de fechas configurable, con desglose por día.	4.1	CSV / PDF	/admin/bitacora/export-accesos-periodo
R-03	Historial de actividad	Bitácora de actividad filtrable por usuario, módulo, tipo de acción y rango de fechas.	4.1	CSV / PDF	/admin/bitacora/export-actividad
R-04	Accesos por módulo	Conteo de acciones agrupadas por módulo del sistema en un periodo.	4.1	CSV / PDF	/admin/bitacora/export-actividad-modulo
R-05	Lecturas por usuario	Historial completo de lecturas de tarjeta de un usuario en un periodo seleccionado.	4.10	CSV / PDF	/admin/tarjetas/reportes/export-csv
R-06	Uso por módulo	Conteo de lecturas de tarjeta	4.10	CSV / PDF	/admin/tarjetas/reportes/export-modulo-csv

	(tarjeta)	agrupadas por módulo del campus y tipo de operación en un periodo.			
R-07	Incidentes y auditoría	Listado de lecturas fallidas, tarjetas bloqueadas y eventos de seguridad relevantes en un periodo.	4.10	CSV / PDF	/admin/tarjetas/reportes/export-incidentes

Apéndice D – Cobertura mínima de pruebas unitarias (PHPUnit)

Cada requisito funcional marcado como Alta/Esencial debe contar con al menos un caso de prueba unitaria en PHPUnit. A continuación se listan las áreas de cobertura mínima obligatoria para el Equipo 6:

Requisito	Área de prueba	Casos mínimos requeridos
RF-01	Registro de usuario	Correo único → error 422. Correo nuevo → usuario creado. Contraseña hasheada (bcrypt). Campos requeridos faltantes → error de validación. Email de verificación enviado.
RF-02	Autenticación	Credenciales correctas → sesión iniciada y redirección a dashboard. Contraseña incorrecta → error, registro en bitácora. Bloqueo tras N intentos fallidos → campo bloqueado=true.
RF-03	Roles y permisos	Asignación de rol a usuario → relación en usuario_rol. Acceso con middleware 'permission:X' sin permiso → 403. Permiso asignado al rol → acceso concedido. Revocación de rol → acceso denegado.
RF-04	Perfil de usuario	Actualización de nombre → persistido. Cambio de contraseña con contraseña actual incorrecta → error. Cambio de contraseña correcto → hash actualizado.
RF-05	Sesiones	Cierre de sesión → sesion.activa=false, termina_at actualizado. Acceso con sesión expirada → redirección a login.
RF-06	Bitácora	Login exitoso → registro en acceso_bitacora con exito=true. Login fallido → registro con exito=false. Creación de usuario → registro en actividad_bitacora.

		Cambio de rol → registro en actividad_bitacora.
RF-10-01	Registro de tarjeta	UID único → tarjeta creada. UID duplicado → error 422. Usuario inexistente → error 404. Asociación correcta a usuario_id.
RF-10-02	Autenticación por tarjeta	Tarjeta activa + UID correcto → sesión iniciada. Tarjeta bloqueada → error 403. UID no registrado → error 404. UID correcto + PIN incorrecto → error 401.
RF-10-05	Bloqueo de tarjeta	Bloquear tarjeta activa → estado='bloqueada'. Usar tarjeta bloqueada en autenticación → denegado. Desbloquear tarjeta → estado='activa'. Evento registrado en bitácora.
RF-10-06	Auditoría de lecturas	Lectura exitosa → registro en tarjeta_lectura con exito=true. Lectura con tarjeta bloqueada → registro con exito=false. Filtros por módulo, usuario y fecha funcionan correctamente.

Apéndice E – Endpoints de API REST (resumen por módulo)

La siguiente tabla resume los endpoints de la API REST implementados y documentados para los módulos 4.1 y 4.10 (base URL: <http://localhost:8000/api>). Autenticación requerida: header X-API-KEY (excepto POST /rfid/auth).

Módulo	Endpoint	Descripción	Método	Auth
4.1	GET /usuarios	Lista paginada de usuarios	GET	X-API-KEY
4.1	POST /usuarios	Crear nuevo usuario	POST	X-API-KEY
4.1	PUT /usuarios/{id}	Actualizar datos del usuario	PUT	X-API-KEY
4.1	DELETE /usuarios/{id}	Baja lógica del usuario	DELETE	X-API-KEY
4.1	POST /usuarios/{id}/toggle-block	Bloquear/desbloquear cuenta	POST	X-API-KEY
4.1	GET /roles	Listar todos los roles	GET	X-API-KEY
4.1	GET /permisos	Listar todos los permisos	GET	X-API-KEY
4.1	GET/POST /rol-permisos	Asignación rol-permiso	GET/POST	X-API-KEY
4.1	GET/POST /usuario-roles	Asignación usuario-rol	GET/POST	X-API-KEY

4.1	GET /sesiones	Historial de sesiones (filtrable)	GET	X-API-KEY
4.1	GET /bitacora/accesos	Bitácora de accesos (filtrable)	GET	X-API-KEY
4.1	GET /bitacora/actividad	Bitácora de actividad (filtrable)	GET	X-API-KEY
4.10	GET /tarjetas	Listar tarjetas (filtrable por estado)	GET	X-API-KEY
4.10	POST /tarjetas	Registrar nueva tarjeta	POST	X-API-KEY
4.10	GET /tarjetas/uid/{uid}	Buscar tarjeta por UID del chip	GET	X-API-KEY
4.10	POST /tarjetas/{id}/bloquear	Bloquear tarjeta con motivo	POST	X-API-KEY
4.10	POST /tarjetas/{id}/desbloquear	Desbloquear tarjeta	POST	X-API-KEY
4.10	GET /tarjeta-lecturas	Historial completo de lecturas	GET	X-API-KEY
4.10	POST /tarjeta-lecturas	Registrar nueva lectura (hardware)	POST	X-API-KEY
4.10	POST /rfid/auth	Autenticar usuario con UID + PIN	POST	PÚBLICA
4.10	GET /rfid/saldo/{uid}	Saldo del monedero por UID	GET	X-API-KEY
4.10	GET /rfid/pedidos/{uid}	Pedidos activos por UID	GET	X-API-KEY
4.10	POST /pedidos/{id}/confirmar-tarjeta	Confirmar entrega con tarjeta	POST	X-API-KEY

Este documento fue elaborado conforme al estándar IEEE Std 830-1998 y los lineamientos del Proyecto Integrador SCD-1011.

Equipo No. 6 – Módulos 4.1 y 4.10 – Parcial 2 – Ingeniería de Software SCD-1011 – Versión 1.1